

An Internship report on
STUDENT MANAGEMENT SYSTEM(CRUD)
in partial fulfillment for the award of the degree of
BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by
Gangumolu Leela Sowmya
(21B91A0595)



S.R.K.R. ENGINEERING COLLEGE (A)
SRKR MARG, CHINNA AMIRAM, BHIMAVARAM-534204, A.P
(Recognized by A.I.C.T.E New Delhi) (Accredited by NBA & NAAC)
(Affiliated to JNTU, KAKINADA)

S.R.K.R ENGINEERING COLLEGE(A)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the Summer Internship Report titled “**STUDENT MANAGEMENT SYSTEM**” is the bonafide work done by Ms. **Gangumolu Leela Sowmya(21B91A0595)** at the end of third year second semester at INTERNATIONAL INSTITUTE OF DIGITAL TECHNOLOGIES, TIRUPATI in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

Department Internship Coordinator

Dean -T & P Cell

Head of the Department

DECLARATION

The Summer Internship Project report entitled “STUDENT MANAGEMENT SYSTEM” has been carried out by me in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in the Department of Computer Science and Engineering, Sagi Rama Krishnam Raju Engineering College (A), Bhimavaram. I hereby declare that this work has not been submitted to any other university/institute for the award of any other degree/diploma.

Roll. No.

Name

Signature

21B91A0595

Gangumolu Leela Sowmya

CERTIFICATE



BLACKBUCK
ENGINEERS

ANDHRA PRADESH STATE COUNCIL OF HIGHER EDUCATION
(A Statutory Body of the Government of A.P.)

Certificate of Completion



Certificate Id: BBAPSCHEIIDT2024ST103126

This is to certify that **GANGUMOLU LEELA SOWMYA**, bearing Reg. No: **21B91A0595**, from **Sagi Rama Krishnam Raju Engineering College, Bhimavaram** of **JNTU Kakinada**, has successfully completed a **Short-term internship** for **8 Weeks** on **Full Stack Development**. This internship was organized by **International Institute of Digital Technologies**, with its industry partner **Blackbuck Engineers**, in association with the **Andhra Pradesh State Council of Higher Education (APSCHE)**.




Anuradha Thota
Chief Executive Officer
Blackbuck Engineers Pvt. Ltd.




Dr. Sundar Balakrishna
Director General
International Institute of Digital Technologies

Date: 24/07/2024 Place: Tirupati, Andhra Pradesh

Table Of Contents

S.NO	Contents	page no
1.	ABSTRACT	1
2.	INTRODUCTION	1
3.	SOFTWARE REQUIREMENTS	2
4.	HARDWARE REQUIREMENTS	2
5.	PROJECT ARCHITECTURE	3
6.	TECHNICAL STACK	5
7.	BACKEND CODE	7
8.	FRONTEND CODE	12
9.	SCREENSHOTS	21

Title:

Student Management System(CRUD)

Abstract:

The Student Management CRUD application is a practical solution designed to streamline the management of student information within an organization. It leverages the capabilities of both Django and React to achieve a seamless integration of backend and frontend technologies, ensuring robust functionality and a dynamic user experience.

Introduction:

Managing student data efficiently is crucial for any educational institution or organization. The Student Management System developed here aims to provide a comprehensive solution for handling student information through CRUD operations: Create, Read, Update, and Delete. This ensures that administrators and educators can easily input, access, modify, and manage student records .

Problem Statement:

The primary challenge addressed by the Student Management System is the effective organization and utilization of student data within an educational setting. Traditional methods of managing student records manually or using disparate systems often lead to inefficiencies and inaccuracies.

SOFTWARE REQUIREMENTS

1. Text Editor : Visual Studio Code
2. Frontend FrameWork: React
3. Backend FrameWork:Django
4. Internet Browser : Google Chrome/Microsoft Edge/Internet Browser

HARDWARE REQUIREMENTS

1. Processor-Intel Core I5
2. RAM-8G

PLAN AND ARCHITECTURE

The architecture of the Student Management System revolves around a user-friendly interface where administrators can input student details via text boxes. This data is then stored securely in a database, facilitating efficient data retrieval and management.

ARCHITECTURE

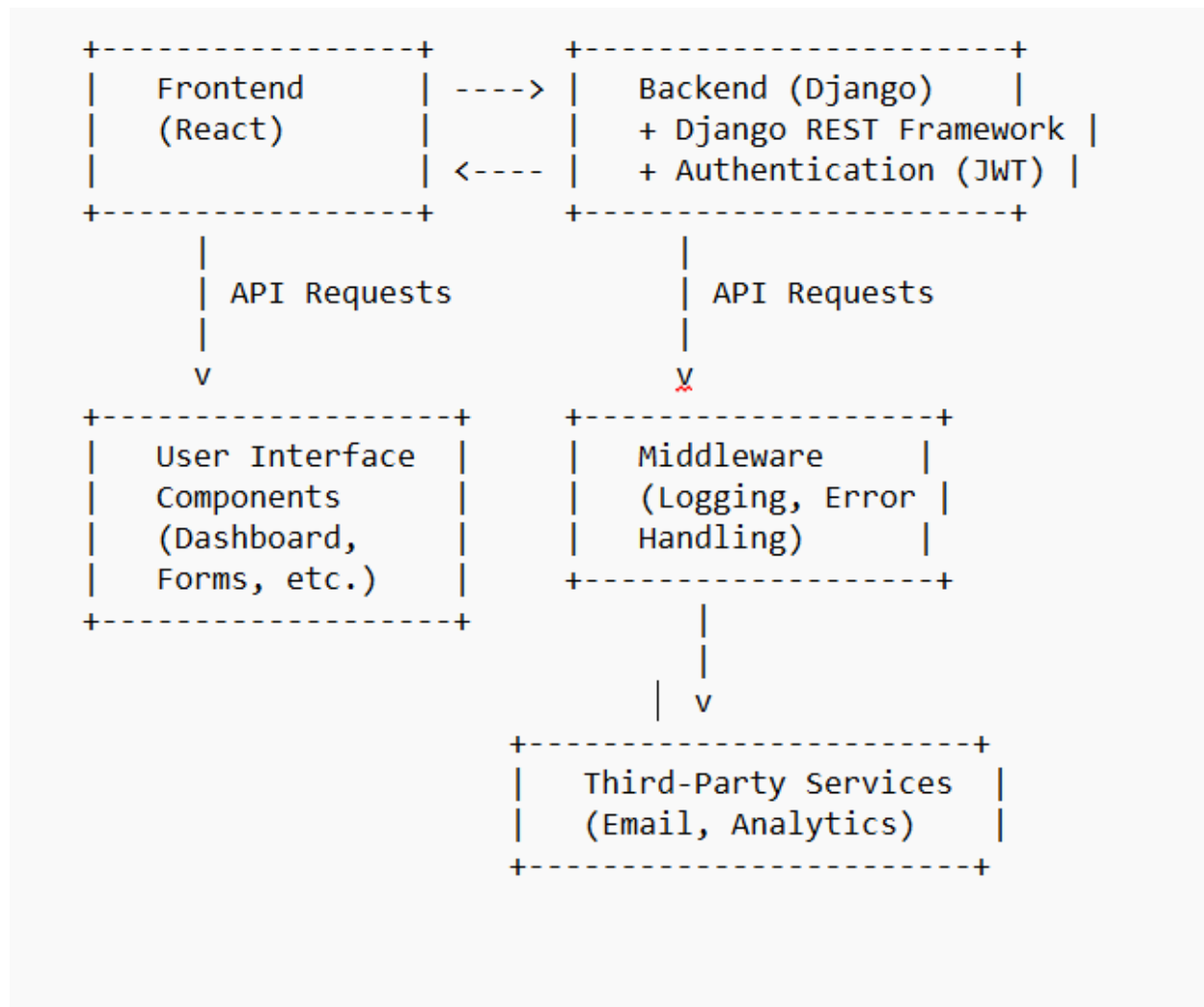


Fig: Flow Of Project

ER DIAGRAM

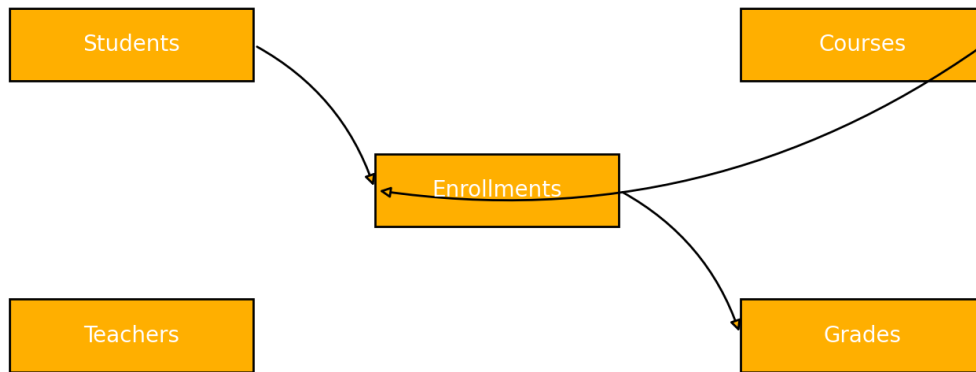


Fig: ER Diagram

Description:

Here is the ER diagram representing the normalized database schema for your Student Management CRUD application. The diagram visually shows the relationships between the Students, Courses, Enrollments, Teachers, and Grades tables.

TECHNICAL STACK REACT

React, also known as **ReactJS**, is a popular and powerful JavaScript library used for building dynamic and interactive user interfaces, primarily for **single-page applications (SPAs)**. It was developed and maintained by Facebook and has gained significant popularity due to its efficient **rendering techniques**, **reusable components**, and active community support.

In this article, we will explore **React Introduction**, **what React is**, its key features, benefits, and why it's a great choice for modern web development.

React is a **JavaScript library** for building **user interfaces (UIs)** on the web. React is a declarative, component based library that allows developers to build reusable UI components and It follows the [Virtual DOM](#) (Document Object Model) approach, which optimizes rendering performance by minimizing DOM updates. React is **fast** and works well with other tools and libraries.

History of React

React was invented by Facebook developers who found the traditional DOM slow. By implementing a virtual DOM, React addressed this issue and gained popularity rapidly. The current stable version of ReactJS is 18.2.0, released on June 14, 2022. The library continues to evolve, introducing new features with each update.

DJANGO

Django is a Python framework that makes it easier to create web sites using Python.

Django takes care of the difficult stuff so that you can concentrate on building your web applications.

Django emphasizes reusability of components, also referred to as DRY (Don't Repeat Yourself), and comes with ready-to-use features like login system, database connection and CRUD operations (Create Read Update Delete).

Django follows the MVT design pattern (Model View Template).

- Model - The data you want to present, usually data from a database.
- View - A request handler that returns the relevant template and content - based on the request from the user.
- Template - A text file (like an HTML file) containing the layout of the web page, with logic on how to display the data.

MVC Pattern

When talking about applications that provides UI (web or desktop), we usually talk about MVC architecture. And as the name suggests, MVC pattern is based on three components: Model, View, and Controller. Check our MVC tutorial [here](#) to know more.

CODE

1.0001_initial.py

Generated by Django 5.0.6 on 2024-06-22 08:50

from django.db import migrations, models

class Migration(migrations.Migration):

initial = True

dependencies = [

]

operations = [

migrations.CreateModel(

name='Students',

fields=[

('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),

('first_name', models.CharField(max_length=100)),

('last_name', models.CharField(max_length=100)),

('age', models.IntegerField()),

('gender', models.CharField(max_length=6)),

('grade', models.CharField(max_length=6)),

('address', models.TextField()),

('contact_number', models.CharField(max_length=15)),

],

),

]

2.admin.py

```
from django.contrib import admin
from .models import Students
# Register your models here.
admin.site.register(Students)
```

3.apps.py

```
from django.apps import AppConfig
class ApiConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'api'
```

4.models.py

```
from django.db import models

# Create your models here.
class Students(models.Model):
    first_name=models.CharField(max_length=100)
    last_name=models.CharField(max_length=100)
    age=models.IntegerField()
    gender=models.CharField(max_length=6)
    grade=models.CharField(max_length=6)
    address=models.TextField()
    contact_number=models.CharField(max_length=15)

    def __str__(self):
        return self.first_name
```

5.serializer.py

```
from rest_framework.serializers import ModelSerializer
```

```
from .models import Students
```

```
class StudentSerializer(ModelSerializer):
```

```
    class Meta:
```

```
        model=Students
```

```
        fields='__all__'
```

5.urls.py

```
from django.urls import path
```

```
from .views import StudentDetailView,StudentListCreateView
```

```
urlpatterns=[
```

```
    path('students/',StudentListCreateView.as_view(),name="student-list-create"),
```

```
    path('students/<int:pk>/',StudentDetailView.as_view(),name="student-detail"),
```

```
]
```

6.views.py

```
from django.shortcuts import render
```

```
# Create your views here.
```

```
from rest_framework import generics
```

```
from .models import Students
```

```
from .serializer import StudentSerializer
```

```
class StudentListCreateView(generics.ListCreateAPIView):
```

```
    queryset=Students.objects.all()
```

```
    serializer_class=StudentSerializer
```

```
class StudentDetailView(generics.RetrieveUpdateDestroyAPIView):
```

```
    queryset=Students.objects.all()
```

```
serializer_class=StudentSerializer
```

7.asgi.py

```
"""
```

ASGI config for backend project.

It exposes the ASGI callable as a module-level variable named ``application``.

For more information on this file, see

<https://docs.djangoproject.com/en/5.0/howto/deployment/asgi/>

```
"""
```

```
import os
```

```
from django.core.asgi import get_asgi_application
```

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'backend.settings')
```

```
application = get_asgi_application()
```

8.urls.py

```
from django.contrib import admin
```

```
from django.urls import path,include
```

```
urlpatterns = [
```

```
    path('admin/', admin.site.urls),
```

```
    path('api/', include('api.urls'))
```

```
]
```

9.wsgi.py

```
import os
```

```
from django.core.wsgi import get_wsgi_application
```

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'backend.settings')
```

```
application = get_wsgi_application()
```

10.manage.py

```
#!/usr/bin/env python
```

```
"""Django's command-line utility for administrative tasks."""
```

```
import os
```

```
import sys
```

```
def main():
```

```
    """Run administrative tasks."""
```

```
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'backend.settings')
```

```
    try:
```

```
        from django.core.management import execute_from_command_line
```

```
    except ImportError as exc:
```

```
        raise ImportError(
```

```
            "Couldn't import Django. Are you sure it's installed and "
```

```
            "available on your PYTHONPATH environment variable? Did you "
```

```
            "forget to activate a virtual environment?"
```

```
        ) from exc
```

```
    execute_from_command_line(sys.argv)
```

```
if __name__ == '__main__':
```

```
    main()
```

CODE

1.app.js

```
import React, {useState, useEffect} from 'react'
import axios from 'axios'
import './main.css'

const App=()=>=>{
  const [students, setStudents] = useState([])
  const [newStudent, setNewStudent] = useState({
    first_name: "",
    last_name: "",
    age: "",
    gender: "",
    grade: "",
    address: "",
    contact_number: ""
  })
  const [selectedStudent, setSelectedStudent] = useState(null)
  const [toView, setToView] = useState({
    first_name: "",
    last_name: "",
    age: "",
    gender: "",
    grade: "",
    address: "",
    contact_number: ""
  })
  const [openView, setOpenview] = useState(true);
  useEffect(() => {
    fetchStudents()
```

```

}, [])

const fetchStudents = () => {
  axios.get('http://127.0.0.1:8000/api/students/')
    .then(response => {
      setStudents(response.data)
    })
    .catch(error => console.error(error))
}

const handleInputChange = (e) => {
  setNewStudent({...newStudent, [e.target.name]:e.target.value})
}

const handleAddStudent = () => {
  axios.post('http://127.0.0.1:8000/api/students/', newStudent)
    .then(response => {
      setStudents([...students, response.data])
      setNewStudent({
        first_name: "",
        last_name: "",
        age: "",
        gender: "",
        grade: "",
        address: "",
        contact_number: ""
      })
    })
    .catch(error => console.error(error))
}

const handleViewClick = async(id) => {
  const response = await axios.get(`http://127.0.0.1:8000/api/students/${id}/`)
  setToView(response.data)
  setOpenview(true)
}

```

```

const handleEditClick = (student) => {
  setSelectedStudent(student)
  setNewStudent(student)
}

const handleUpdateStudent = (id) => {
  axios.put(`http://127.0.0.1:8000/api/students/${selectedStudent.id}/`, newStudent)
    .then(response => {
      fetchStudents();
      setNewStudent({
        first_name: "",
        last_name: "",
        age: "",
        gender: "",
        grade: "",
        address: "",
        contact_number: ""
      });
    })
    .catch(error => console.error(error))
}

const handleCancelUpdateStudent = () => {
  setSelectedStudent(null)
  setNewStudent({
    first_name: "",
    last_name: "",
    age: "",
    gender: "",
    grade: "",
    address: "",
    contact_number: ""
  })
}

```

```

const handleDeleteStudent = (id) => {
  axios.delete(`http://127.0.0.1:8000/api/students/${id}/`, )
    .then(response => {
      fetchStudents();
      setNewStudent({
        first_name: "",
        last_name: "",
        age: "",
        gender: "",
        grade: "",
        address: "",
        contact_number: ""
      });
    })
    .catch(error => console.error(error))
  }
  return(
    <div className='app-container'>
      <h1>Student Management System</h1>
      {/*Form Container*/}
      <div className='form-container'>
        <div className='form-inputs'>
          <input type='text' name='first_name' placeholder='First name'
            value={newStudent.first_name} onChange={handleInputChange} />
          <input type='text' name='last_name' placeholder='Last name'
            value={newStudent.last_name} onChange={handleInputChange} />
          <input type='text' name='age' placeholder='Age' value={newStudent.age}
            onChange={handleInputChange} />
          <input type='text' name='gender' placeholder='Gender' value={newStudent.gender}
            onChange={handleInputChange} />
          <input type='text' name='grade' placeholder='Grade' value={newStudent.grade}
            onChange={handleInputChange} />
          <textarea name='address' placeholder='Address' value={newStudent.address}
            onChange={handleInputChange} />

```

```

    <input type='text' name='contact_number' placeholder='Contact Number'
value={newStudent.contact_number} onChange={handleInputChange} />

    <div className='form-buttons'>
    {
      selectedStudent ? (
        <>
          <button onClick={handleUpdateStudent}>Update</button>
          <button onClick={handleCancelUpdateStudent}>Cancel</button>
        </>
      ) : (
        <button onClick={handleAddStudent}>Add New Student</button>
      )
    }
  </div>
</div>

  { /* Student List */ }
  <ul className='student-list'>
  {
    students.map(student => (
      <li key={student.id}>
        <div>
          <strong>{student.first_name} {student.last_name}</strong>
        </div>
        <div className='actions'>
          <button className='view' onClick={() => handleViewClick(student.id)}
>View</button>
          <button className='edit' onClick={() => handleEditClick(student)}>Edit</button>
          <button className='delete' onClick={() =>
handleDeleteStudent(student.id)}>Delete</button>
        </div>
      </li>
    ))
  }

```

```

    }
  </ul>

  { /* Single View */
    {openView && (
      <>
        <div className='outer-box'>
          <strong>{toView.first_name}{toView.last_name}</strong>
          <br />
          <span>Age : {toView.age}</span>
          <br />
          <span>Gender : {toView.gender}</span>
          <br />
          <span>Grade : {toView.grade}</span>
          <br />
          <span>Address : {toView.address}</span>
          <br />
          <span>Contact Line : {toView.contact_number}</span>
          <br />
        </div>
        <button onClick={() => setOpenview(false)}>Close</button>
      </>
    )}
  </div>
)
}

export default App

```

2.index.js

```

import React from 'react';
import ReactDOM from 'react-dom';

```

```
import App from './app';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
root.render(<App />);
```

3.main.css

```
body{
```

```
  font-family: 'Arial', sans-serif;
```

```
  margin: 0;
```

```
  padding: 0;
```

```
  background-color: #f5f5f5;
```

```
}
```

```
.app-container {
```

```
  max-width: 800px;
```

```
  margin: 2rem auto;
```

```
  padding: 20px;
```

```
  background-color: #fff;
```

```
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
```

```
  border-radius: 8px;
```

```
}
```

```
h1 {
```

```
  text-align: center;
```

```
  color: #333;
```

```
}
```

```
.form-container {
```

```
  display: flex;
```

```
  justify-content: space-between;
```

```
  margin-bottom: 20px;
```

```
}
```

```
.form-inputs,
```

```
.form-buttons {
```

```

    width: 48%;
}

.form-inputs input,
.form-inputs textarea {
    width: 100%;
    margin-bottom: 10px;
    padding: 8px;
    font-size: 16px;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box;
}

.form-buttons button {
    padding: 10px;
    margin-top: 10px;
    cursor: pointer;
    background-color: #4caf50;
    color: #fff;
    border: none;
    font-size: 16px;
    border-radius: 4px;
    transition: background-color 0.3s;
}

.form-buttons button:hover {
    background-color: #45a049;
}

.student-list {
    list-style: none;
    padding: 0;
}

.student-list li {
    background-color: #fff;

```

```

border: 1px solid #ddd;
border-radius: 4px;
padding: 15px;
margin-bottom: 15px;
display: flex;
justify-content: space-between;
align-items: center;
transition: box-shadow 0.3s;
}

.student-list li:hover {
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

.student-list li div {
    width: 75%;
}

.student-list li .actions {
    width: 20%;
    display: flex;
    justify-content: space-between;
}

.student-list button {
    padding: 8px;
    cursor: pointer;
    font-size: 14px;
    transition: background-color 0.3s;
}

.student-list button.edit {
    background-color: #2196F3;
    color: #fff;
}

.student-list button.delete {
    background-color: #f44336;

```

```
    color: #fff;
}
.student-list button:hover {
    background-color: #ddd;
}
```

SCREENSHOTS

Fig1: Add Student

The screenshot shows a web browser window with the title 'Site administration | Django site'. The address bar shows 'localhost:3000'. The page displays a form titled 'Student Management System'. The form contains the following fields: First name, Last name, Age, Gender, Grade, Address, and Contact Number. Below the fields are two buttons: 'Update' and 'Cancel'. The browser's taskbar at the bottom shows the system clock as 15:52 on 12-07-2024.

Student Management System

First name

Last name

Age

Gender

Grade

Address

Contact Number

Update Cancel

Fig2: Adding Student

The screenshot shows a web browser window with the title 'Select students to change | Django site'. The address bar shows 'localhost:3000'. The page displays a form titled 'Student Management System'. The form contains the following fields: First name (Leela Sowmya), Last name (Gangumolu), Age (21), Gender (Female), Grade (A+), Address (Bhimavaram), and Contact Number (9121121212). Below the fields is a button: 'Add New Student'. The browser's taskbar at the bottom shows the system clock as 15:56 on 12-07-2024.

Student Management System

Leela Sowmya

Gangumolu

21

Female

A+

Bhimavaram

9121121212

Add New Student

Fig3: After Adding Student

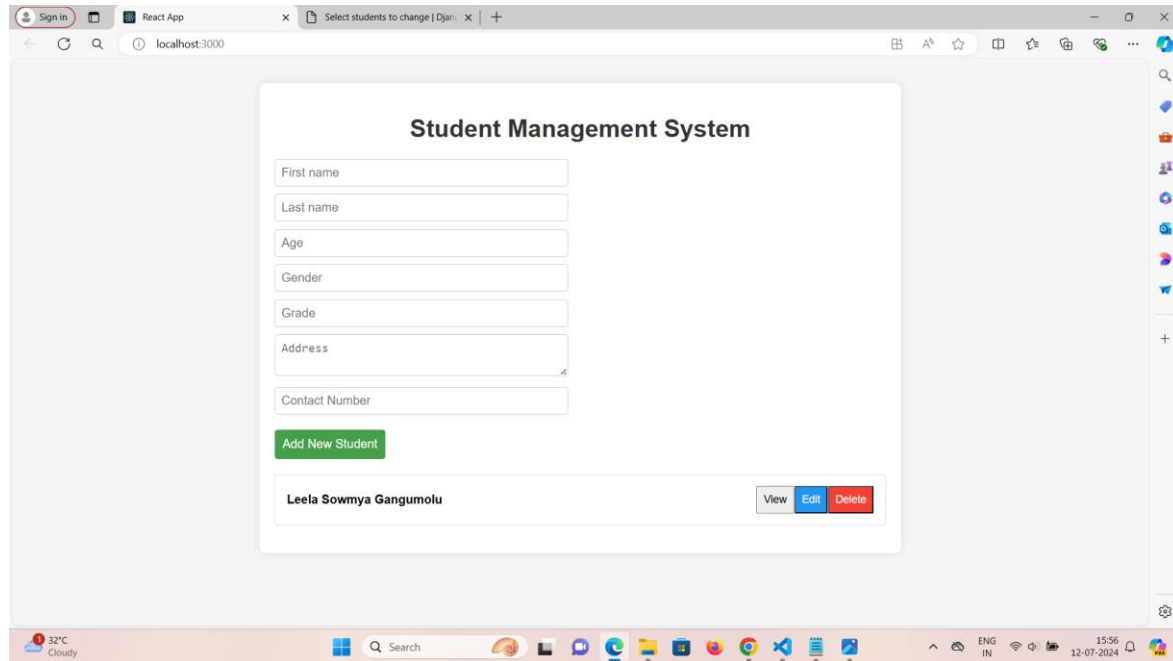


Fig4: Viewing Student

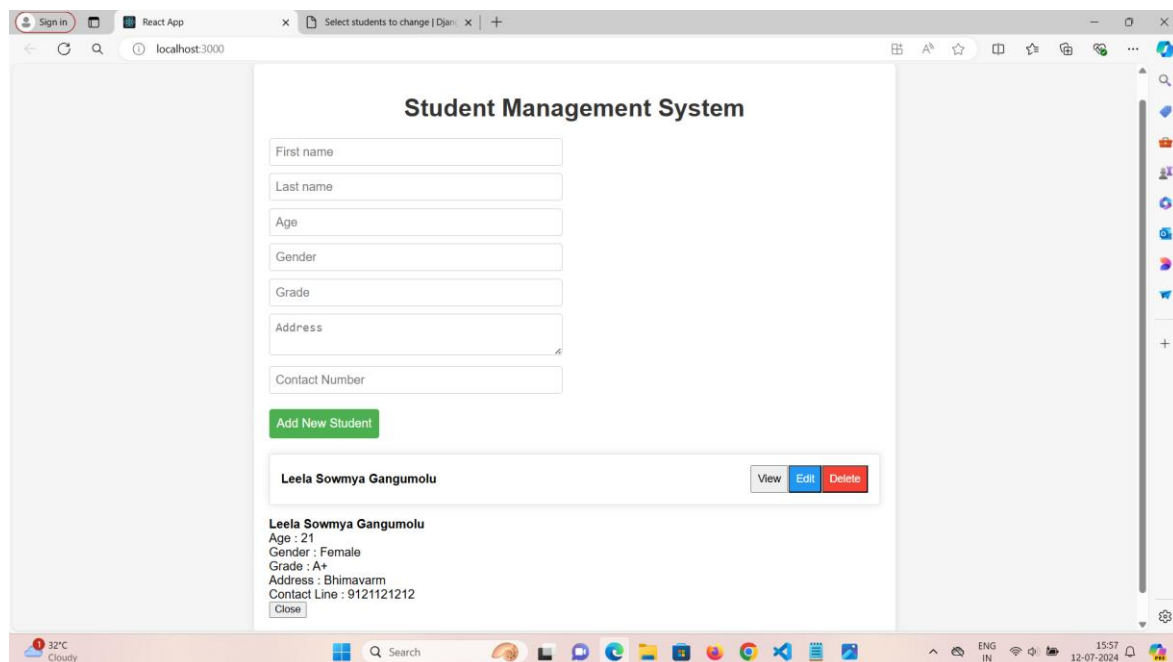


Fig5: Update Student

The screenshot shows a web browser window with the URL 'localhost:3000'. The page title is 'Student Management System'. The form contains the following fields: Name (Leela Sowmya), Surname (Gangumolu), Age (21), Gender (Female), Grade (A+), Address (Bhimavarm), and Contact Line (9121121212). Below the form are 'Update' and 'Cancel' buttons. A summary box displays the student's name 'Leela Sowmya Gangumolu' with 'View', 'Edit', and 'Delete' buttons. A detailed view of the student's information is shown below, including Age, Gender, Grade, Address, and Contact Line, with a 'Close' button.

Student Management System

Leela Sowmya
Gangumolu
21
Female
A+
Bhimavarm
9121121212

Update Cancel

Leela Sowmya Gangumolu View Edit Delete

Leela Sowmya Gangumolu
Age : 21
Gender : Female
Grade : A+
Address : Bhimavarm
Contact Line : 9121121212
Close

Fig6: Edit/Delete

The screenshot shows a web browser window with the URL 'localhost:3000'. The page title is 'Student Management System'. The form contains the following fields: Age, Gender, Grade, Address, and Contact Number. Below the form is an 'Add New Student' button. A list of students is displayed, each with a 'View', 'Edit', and 'Delete' button. The first student is 'Leela Sowmya Gangumolu'. The second student is 'Prudhvi Gandrothu'. The third student is 'srihitha galala'. A detailed view of the student's information is shown below, including Age, Gender, Grade, Address, and Contact Line, with a 'Close' button.

Age
Gender
Grade
Address
Contact Number

Add New Student

Leela Sowmya Gangumolu View Edit Delete

Prudhvi Gandrothu View Edit Delete

srihitha galala View Edit Delete

Prudhvi Gandrothu
Age : 20
Gender : Male
Grade : A
Address : Munganda
Contact Line : 9014821844
Close

Fig7: DataBase of Students

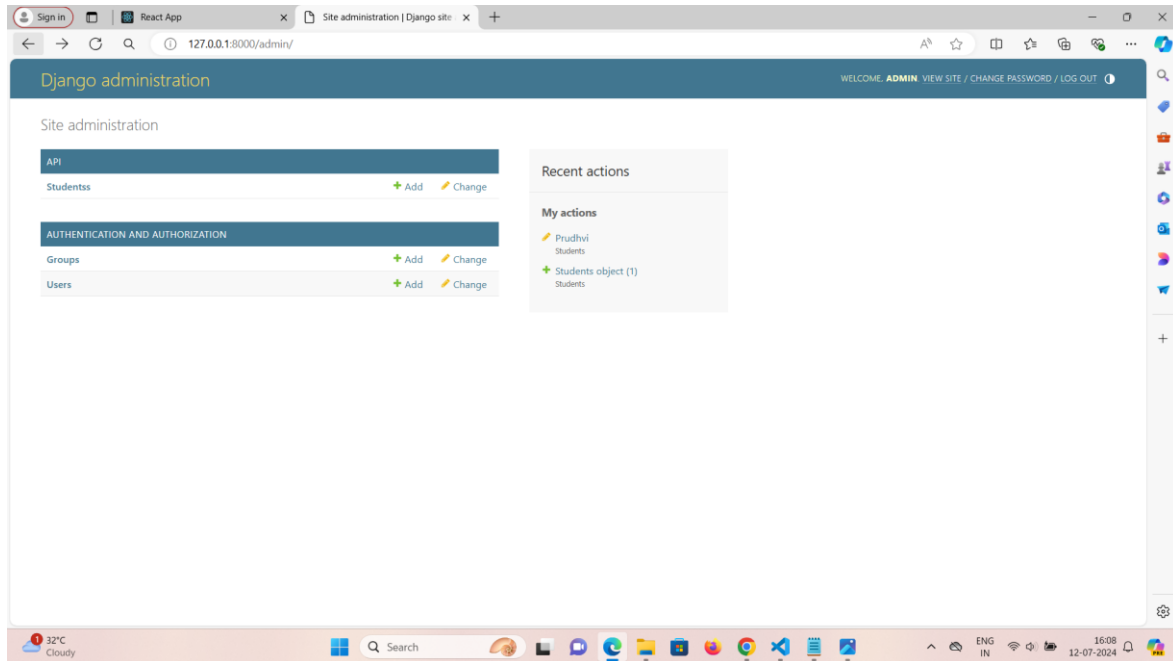


Fig8: DataBase of Students

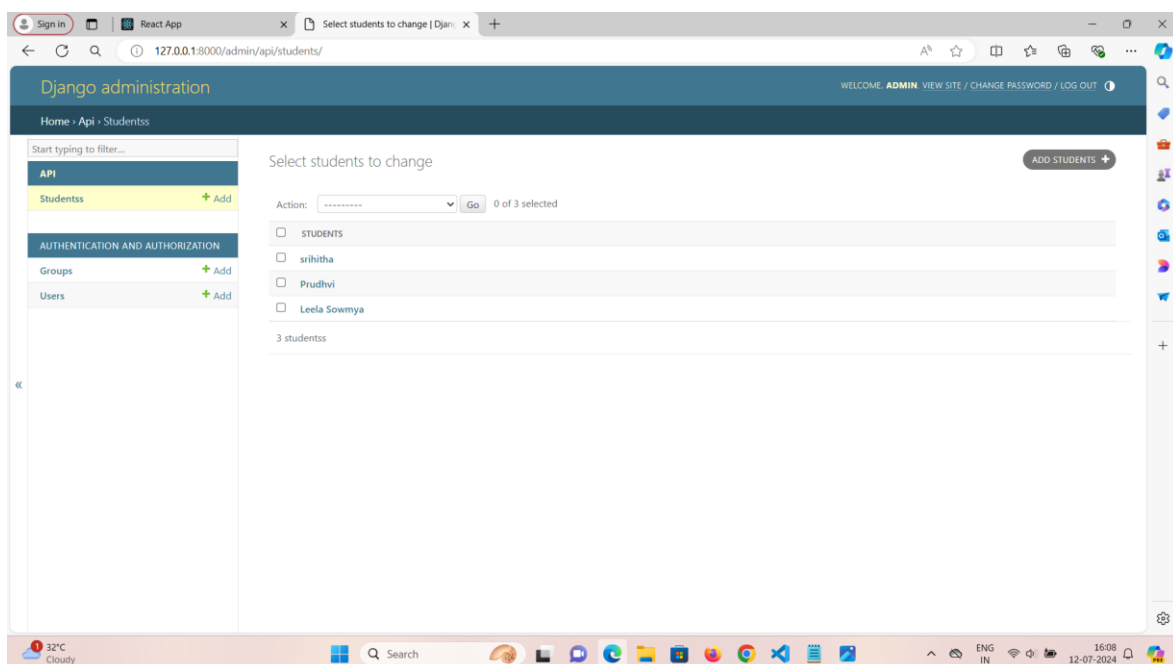


Fig9: Adding Students to DataBase.

Home > Api > Studentss > Add students

Start typing to filter...

API

Studentss [+ Add](#)

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Users [+ Add](#)

✓ The students "Leela Sowmya" was changed successfully. You may add another students below.

Add students

First name:

Last name:

Age:

Gender:

Grade:

Address:

Contact number:

[SAVE](#) [Save and add another](#) [Save and continue editing](#)

Fig9: Added Successfully.

Home > Api > Studentss > Add students

Start typing to filter...

API

Studentss [+ Add](#)

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Users [+ Add](#)

✓ The students "Leela Sowmya" was changed successfully. You may add another students below.

Add students

First name:

Last name:

Age:

Gender:

Grade:

Address:

Contact number:

[SAVE](#) [Save and add another](#) [Save and continue editing](#)

CONCLUSION

In conclusion, the Student Management CRUD application offers a straightforward yet powerful solution for educational institutions to manage student information effectively.